

OS Practical 9

File copy using open, read, write, lseek, close

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s source destination\n", argv[0]);
        return 1;
    }

    int src = open(argv[1], O_RDONLY);
    if (src == -1) {
        perror("open source");
        return 1;
    }

    int dest = open(argv[2],
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (dest == -1) {
        perror("open destination");
        close(src);
        return 1;
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    clock_t start = clock();

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0) {
        ssize_t bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read) {
            perror("write");
            close(src);
            close(dest);
            return 1;
        }
    }
}
```

```

    clock_t end = clock();

    if (bytes_read == -1) {
        perror("read");
    }

    printf("Low-level copy time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);

    close(src);
    close(dest);

    return 0;
}

```

```

nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ gcc low_copy.c -o low_copy
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ echo "Hello Linux System Programming" > input.txt
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ ./low_copy input.txt output.txt
Low-level copy time: 0.000062 seconds
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ cat output.txt
Hello Linux System Programming
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$

```

Part 2: Copy using fopen, fread, fwrite

```

#include <time.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s source destination\n", argv[0]);
        return 1;
    }

    FILE *src = fopen(argv[1], "rb");

    if (src == NULL) {
        perror("fopen source");
        return 1;
    }

    FILE *dest = fopen(argv[2], "wb");

    if (dest == NULL) {
        perror("fopen destination");
        fclose(src);
        return 1;
    }

    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    clock_t start = clock();

    while ((bytes_read =
        fread(buffer, 1, BUFFER_SIZE, src)) > 0) {

        fwrite(buffer, 1, bytes_read, dest);
    }

    clock_t end = clock();

    printf("Standard library copy time: %.6f seconds\n",
        (double)(end - start) / CLOCKS_PER_SEC);

    fclose(src);
    fclose(dest);

    return 0;
}

```

```
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ gcc std_copy.c -o std_copy
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ ./std_copy input.txt output2.txt
Standard library copy time: 0.000023 seconds
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ cat output2.txt
Hello Linux System Programming
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ dd if=/dev/zero of=bigfile.bin bs=1M count=500
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 1.9454 s, 270 MB/s
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ time ./low_copy bigfile.bin low_output.bin
Low-level copy time: 3.821441 seconds

real    0m3.920s
user    0m0.092s
sys     0m3.756s
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$ time ./std_copy bigfile.bin std_output.bin
Standard library copy time: 4.941223 seconds

real    0m4.952s
user    0m0.153s
sys     0m4.797s
nishanth@LAPTOP-6Q3ON07H:~/ospractical9$
```

Part 3: Redirect standard input/output using dup2()

```
GNU nano 2.7.1
int main() {

    int input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1) {
        perror("open input");
        return 1;
    }

    int output_fd = open("redirect_output.txt",
                        O_WRONLY | O_CREAT | O_TRUNC,
                        0644);

    if (output_fd == -1) {
        perror("open output");
        close(input_fd);
        return 1;
    }

    /* Redirect stdin to input.txt */
    if (dup2(input_fd, STDIN_FILENO) == -1) {
        perror("dup2 stdin");
        return 1;
    }

    /* Redirect stdout to redirect_output.txt */
    if (dup2(output_fd, STDOUT_FILENO) == -1) {
        perror("dup2 stdout");
        return 1;
    }

    close(input_fd);
    close(output_fd);

    char buffer[256];

    /*
     * scanf/fgets now reads from input.txt
     * because stdin was redirected.
     */
    while (fgets(buffer, sizeof(buffer), stdin) != NULL) {
        printf("Received: %s", buffer);
    }

    return 0;
}
```

```
nishanth@LAPTOP-6Q30N07H:~/ospractical9$ nano redirect.c
nishanth@LAPTOP-6Q30N07H:~/ospractical9$ gcc redirect.c -o redirect
nishanth@LAPTOP-6Q30N07H:~/ospractical9$ ./redirect
nishanth@LAPTOP-6Q30N07H:~/ospractical9$ cat redirect_output.txt
Received: Hello Linux System Programming
nishanth@LAPTOP-6Q30N07H:~/ospractical9$
```